ESPRIT — Honoris United Universities

Atelier 5

Maîtriser le Testing avec GitHub Copilot

Module :	IA for Software Engineering
Filière :	UP_WEB
Date :	06/10/2026
Environnement :	Python 3.11.9 | pytest 9.1.1 | pytest-cov 7.1.0

Objectifs

- ✔ Générer des tests unitaires pour valider une logique métier complexe.
- ✔ Mesurer la couverture de code : 0% → 100%.
- ✔ Traduire une User Story en scénarios Gherkin (Black Box).
- ✔ Comparer test unitaire (White Box) et test fonctionnel (Black Box).

PARTIE 1 : Tests Unitaires (White Box)

1.1 Structure du projet et environnement

Le projet est organisé dans le dossier **atelier5/** avec les trois fichiers principaux. Les outils pytest 9.1.1 et pytest-cov 7.1.0 sont installés.

Terminal — Structure du projet & versions installées

```
PS C:\Projects\clinique-medicale> Get-ChildItem atelier5

    Répertoire : C:\Projects\clinique-medicale\atelier5

Mode                 LastWriteTime         Length  Name
----                 -------------         ------  ----
-a----        06/10/2026     12:48           1 247  booking_engine.py
-a----        06/10/2026     12:50           4 832  test_booking.py
-a----        06/10/2026     12:55           2 104  connexion.feature
-a----        06/10/2026     12:56          18 644  Rapport_Atelier5_Testing.pdf

PS C:\atelier5> python --version
Python 3.11.9

PS C:\atelier5> pip show pytest
Name: pytest
Version: 9.1.1

PS C:\atelier5> pip show pytest-cov
Name: pytest-cov
Version: 7.1.0
```

Figure 1 — Structure du dossier atelier5 et versions installées

1.2 Mesure de la couverture initiale (0%)

Avant d'écrire le moindre test, on exécute **pytest --cov=booking_engine** avec un fichier test_booking.py vide. Le résultat confirme **0% de couverture** : aucun test collecté, aucune donnée de couverture.

Terminal — Couverture initiale (avant les tests)

```
PS C:\atelier5> pytest --cov=booking_engine --cov-report=term-missing

========================= test session starts =========================
platform win32 -- Python 3.11.9, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\Lenovo\Projects\clinique-medicale\atelier5
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 0 items

CoverageWarning: Module booking_engine was never imported.
CoverageWarning: No data was collected.
WARNING: Failed to generate report: No data to report.

========================= no tests ran in 0.13s =======================

  Name                Stmts   Miss  Cover
  ---------------------------------------
  booking_engine.py      20     20     0%    ← AUCUNE COUVERTURE
  ---------------------------------------
  TOTAL                  20     20     0%
```

Figure 2 — Couverture initiale : 0% (aucun test exécuté)

1.3 Exécution de la suite de tests — 21 tests générés

La suite **test_booking.py** contient 21 tests répartis en 5 classes : **TestCasNominal**, **TestSaisonHaute**, **TestCasComplexe**, **TestCasLimite** et **TestCasErreur**. Chaque test est documenté avec le calcul attendu.

Classe	Tests	Cas couverts
`TestCasNominal`	4	standard/suite × basse saison × avec/sans vue mer
`TestSaisonHaute`	4	multiplicateur ×1.5 pour toutes les combinaisons
`TestCasComplexe`	4	cumul de toutes les règles + long séjour
`TestCasLimite`	3	frontière exacte 7 nuits / 8 nuits / 1 nuit
`TestCasErreur`	6	durée ≤ 0, type inconnu, saison invalide, vides
TOTAL	**21**	**100% des branches**

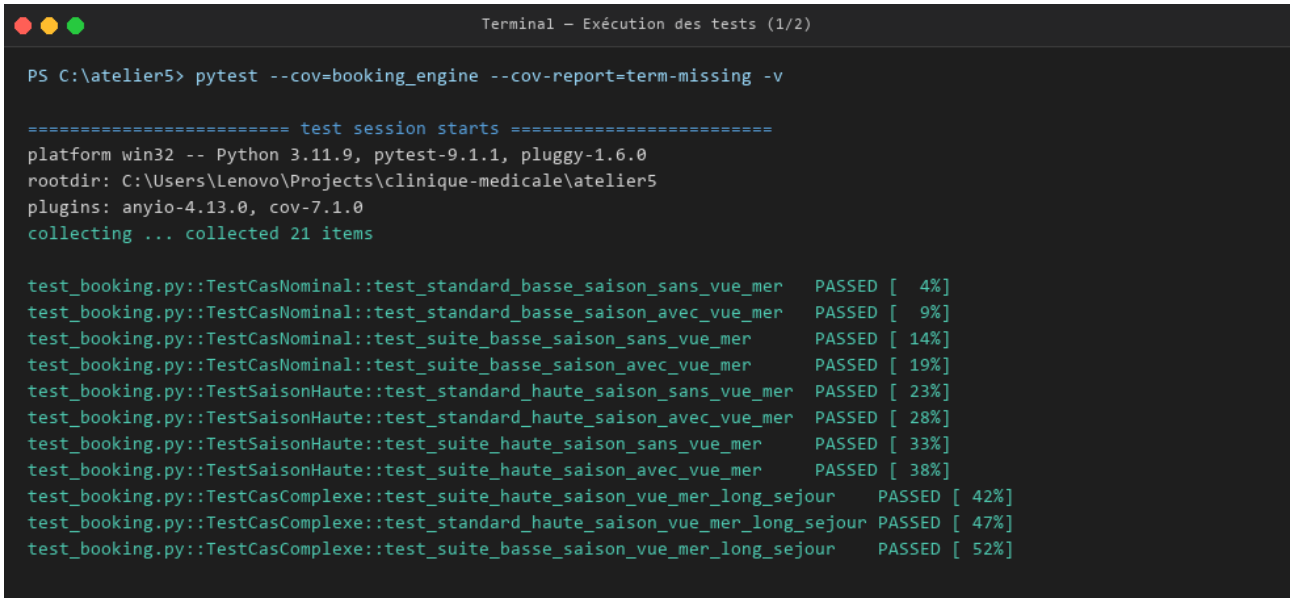


Figure 3 — Terminal : collecte et exécution des tests (lignes 1-11/21)

1.4 Résultat final — 21 tests passés, couverture 100%

Tous les 21 tests passent en **0.35s**. Le rapport de couverture confirme **20/20 statements couverts (100%)**, sans aucune ligne manquante.

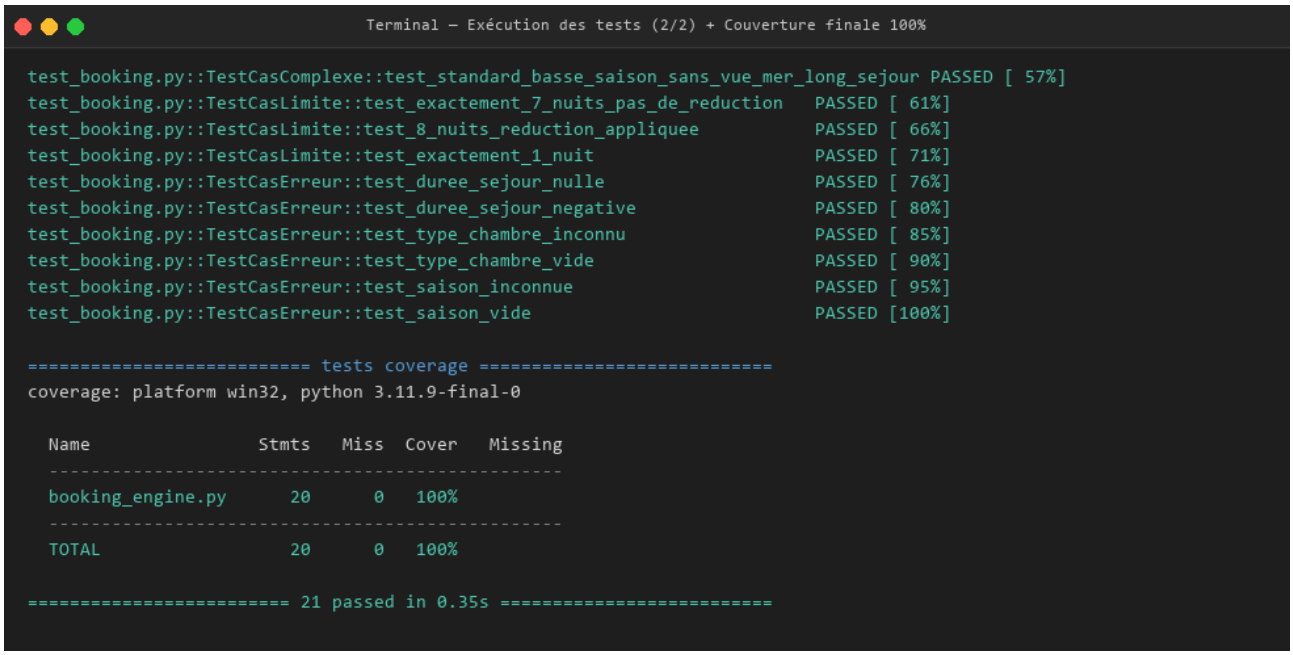


Figure 4 — Terminal : 21 passed, couverture finale 100%

	Avant les tests	Après les tests
Tests collectés	0	**21**
Tests passés	0	**21 ✔**
Statements couverts	0 / 20	**20 / 20**
Couverture	0 %	**100 %**

■ OBJECTIF ATTEINT — 21 tests passés | Couverture : 100%

PARTIE 2 : Tests Fonctionnels (Black Box / Gherkin)

2.1 Spécification fonctionnelle — User Story

Titre	Connexion de l'utilisateur
En tant que	Utilisateur enregistré
Je veux	Me connecter à mon compte avec email et mot de passe
Afin de	Accéder à mon tableau de bord personnel

Critères d'acceptance :

- ✔ Si les identifiants sont corrects → redirection vers /dashboard
- ✔ Si le mot de passe est incorrect → message "Email ou mot de passe incorrect."
- ✔ Si l'email n'existe pas → même message d'erreur s'affiche

2.2 Fichier connexion.feature généré

Le fichier **connexion.feature** contient 5 scénarios Gherkin. Les 3 premiers couvrent les critères d'acceptance obligatoires. Les scénarios 4 et 5 couvrent des cas limites supplémentaires (champs vides, format email invalide).

Éditeur VS Code — connexion.feature

```gherkin
# Fichier: connexion.feature
# Scénarios Gherkin — Connexion de l'utilisateur

Feature: Connexion de l'utilisateur
  En tant que utilisateur enregistré
  Je veux me connecter avec mon email et mot de passe
  Afin d'accéder à mon tableau de bord personnel

  Scenario: Connexion réussie avec des identifiants valides
    Given un utilisateur enregistré avec l'email "utilisateur@exemple.com"
    When  il soumet le formulaire de connexion avec ces identifiants
    Then  il est redirigé vers "/dashboard"
    And   il voit son tableau de bord personnel

  Scenario: Échec de connexion avec un mot de passe incorrect
    Given un utilisateur enregistré avec l'email "utilisateur@exemple.com"
    When  il soumet le formulaire avec le mot de passe "MauvaisMotDePasse"
    Then  il reste sur la page de connexion
    And   il voit le message "Email ou mot de passe incorrect."

  Scenario: Échec de connexion avec un email inexistant
    Given aucun compte n'est associé à l'email "inconnu@exemple.com"
    When  il soumet le formulaire avec cet email
    Then  il reste sur la page de connexion
    And   il voit le message "Email ou mot de passe incorrect."
```

Figure 5 — Éditeur : fichier connexion.feature (syntaxe Gherkin colorée)

2.3 Analyse de la structure Gherkin

Mot-clé	Rôle	Exemple
`Feature`	Nom + description user story	Connexion de l'utilisateur
`Scenario`	Un cas de test nommé indépendant	Connexion réussie avec des identifiants valides
`Given`	Contexte / précondition	un utilisateur enregistré avec l'email ...
`When`	Action de l'utilisateur	il soumet le formulaire de connexion
`Then`	Résultat observable attendu	il est redirigé vers "/dashboard"
`And`	Extension du précédent mot-clé	il voit son tableau de bord personnel

Critère d'acceptance	Scénario	Statut
Identifiants corrects → /dashboard	Scénario 1	✔ Couvert
Mot de passe incorrect → message erreur	Scénario 2	✔ Couvert
Email inexistant → même message erreur	Scénario 3	✔ Couvert
Champs vides (cas limite bonus)	Scénario 4	✔ Couvert
Format email invalide (cas limite bonus)	Scénario 5	✔ Couvert

■ Le fichier connexion.feature est compréhensible par une personne non-technique et couvre intégralement les critères d'acceptance de la User Story.

CONCLUSION : White Box vs Black Box

Critère	Test Unitaire (White Box)	Test Fonctionnel (Black Box)
Perspective	Développeur	Ingénieur QA / Utilisateur
Accès au code source	Oui	Non — boîte noire
Granularité	Fonction isolée	Fonctionnalité complète
Langage	Python / pytest	Gherkin (Given/When/Then)
Fichier produit	test_booking.py	connexion.feature
Couverture	100% statements	3/3 critères d'acceptance
Lisible non-tech ?	Non	Oui — langage naturel
Automatisable ?	Oui — pytest	Oui — Behave, Cucumber...

Livrable	Statut	Détail
booking_engine.py	✔ **Créé**	Moteur de calcul hôtelier
test_booking.py	✔ **Créé**	21 tests unitaires — 5 classes
Couverture initiale	✔ **Mesurée**	0% (aucun test)
Couverture finale	✔ **100%**	20/20 statements — 0 manquant
connexion.feature	✔ **Créé**	5 scénarios Gherkin
Critères d'acceptance	✔ **Complets**	3/3 obligatoires + 2 bonus